

Not a Reading Bug

17 Reports, 4 Mechanisms



I Read the Top 20 IDOR Reports on HackerOne.

30-second version: I pulled the 20 highest-upvoted IDOR reports on HackerOne. Three are no longer publicly readable, so the dataset is 17...

Abhishek meena

Published August 15, 2026

Free: No

[Read on Freedium](#) · [original](#)

Contents

I Read the Top 20 IDOR Reports on HackerOne. Nearly Half of Them Delete Data Instead of Stealing It.	3
Free food, paid for by a stranger	3
What I found in the top 20	4
Where these IDORs actually live	5
The four mechanisms	5
Mechanism 1: Your request, their object	6
Mechanism 2: Two identifiers, never compared	7
Mechanism 3: The boundary is a parameter	9
Mechanism 4: The reference that scales	10
From one invoice to every account: the escalation ladder	10
The report that made it click	11
5 questions to run on every endpoint	12
1. Whose object is in this request?	12
2. What identifies the target — and does anything compare them?	12
3. Is the boundary itself a parameter?	13
4. Can I predict or resolve the reference?	13
5. What does this object unlock one level up?	13
What to remember	13
Sources	14

I Read the Top 20 IDOR Reports on HackerOne. Nearly Half of Them Delete Data Instead of Stealing It.

30-second version: I pulled the 20 highest-upvoted IDOR reports on HackerOne. Three are no longer publicly readable, so the dataset is **17 reports**: 5 critical, 9 high, 3 medium.

Seven live behind GraphQL APIs. Eight let the attacker **delete or modify** another user's data — this is not a read-only bug class.

Every one of the 17 comes down to four mechanisms: the object reference swapped inside an authorized request, two identifiers the server never compares, a trust boundary passed in as a parameter, and a reference predictable enough to enumerate.

Here's the map, the ladder from data leak to account takeover, and the five questions that find all four.

Free food, paid for by a stranger

A researcher orders food on Yelp. At checkout, Grubhub asks which saved card to use. The researcher intercepts the request to `/checkout/transaction_platform` and swaps one value: `credit_card_id`. Not their card. Someone else's.

The order goes through. The meal is paid for with a stranger's saved credit card, and the attacker never sees a card number. Yelp's own disclosure noted the flaw potentially affected every card saved on the platform — more than 1.5 million of them. ([Yelp #391092](https://hackerone.com/reports/391092) (<https://hackerone.com/reports/391092>), 213 upvotes, critical)

That report is the twelfth most-upvoted IDOR on HackerOne. I pulled the top 20 and read every one I could open, because I wanted to answer one question: **what separates the IDORs that pay \$10,000 from the ones that get closed as a duplicate?**

The pattern I expected — change a number in a URL, read someone's data — was there. But it wasn't the main story. The main story surprised me.

What I found in the top 20

I started from the [hackerone-reports](https://github.com/reddelexc/hackerone-reports) (<https://github.com/reddelexc/hackerone-reports>) repository, took the top 20 IDOR entries by upvotes, and fetched each report.

The raw list needed cleaning first:

Top IDOR reports from HackerOne:

1. [IDOR to add secondary users in www.paypal.com/businessmanage/users/api/v1/users](#) to PayPal - 786 upvotes, \$10500
2. [IDOR - Delete all Licenses and certifications from users account using CreateOrUpdateHackerCertification GraphQL query](#) to HackerOne - 388 upvotes, \$0
3. [IDOR allow access to payments data of any user](#) to Nord Security - 387 upvotes, \$0
4. [Insecure Direct Object Reference \(IDOR\) - Delete Campaigns](#) to HackerOne - 346 upvotes, \$0
5. [idor allows you to delete photos and album from a gallery](#) to Pornhub - 266 upvotes, \$1500
6. [Singapore - Account Takeover via IDOR](#) to Starbucks - 259 upvotes, \$0
7. [Insecure Direct Object Reference \(IDOR\) Allows Viewing Private Report Details via /bugs.json Endpoint](#) to HackerOne - 255 upvotes, \$0
8. [IDOR allows any user to edit others videos](#) to Pornhub - 248 upvotes, \$1500
9. [IDOR: Account Deletion via Session Misbinding - Attacker Can Delete Victim Account](#) to Mozilla - 235 upvotes, \$0
10. [An IDOR that can lead to enumeration of a user and disclosure of email and phone number within cashier](#) to Unikrn - 229 upvotes, \$3000
11. [IDOR delete any Tickets on ads.tiktok.com](#) to TikTok - 216 upvotes, \$0
12. [I.D.O.R To Order,Book,Buy,reserve On YELP FOR FREE \(UNAUTHORIZED USE OF OTHER USER'S CREDIT CARD\)](#) to Yelp - 213 upvotes, \$0
13. [IDOR vulnerability in unreleased HackerOne Copilot feature](#) to HackerOne - 208 upvotes, \$0
14. [IDOR allows an attacker to modify the links of any user](#) to Reddit - 206 upvotes, \$0
15. [IDOR when editing users leads to Account Takeover without User Interaction at CrowdSignal](#) to Automattic - 202 upvotes, \$0
16. [IDOR on GraphQL queries BillingDocumentDownload and BillDetails](#) to Shopify - 180 upvotes, \$5000
17. [IDOR leads to Edit Anyone's Blogs / Websites](#) to Automattic - 176 upvotes, \$0
18. [IDOR in the https://market.semrush.com/](#) to Semrush - 172 upvotes, \$0
19. [IDOR Vulnerability at AddTagToAssets operation name](#) to HackerOne - 172 upvotes, \$0
20. [Getting access of mod logs from any public or restricted subreddit with IDOR vulnerability](#) to Reddit - 157 upvotes, \$5000

- **3 reports are no longer publicly readable** — they now redirect to a login page (Nord Security #751577, Pornhub #380410, Pornhub #681473).
- **17 reports remained** — the real dataset: 5 critical, 9 high, 3 medium.

Then the numbers started talking:

Finding	Count
Rated high or critical	14 of 17
Live behind a GraphQL API	7 of 17
Let the attacker delete or modify someone's data	8 of 17
Swap the reference in a request body , not a URL	12 of 17

Two of those rows broke what I thought I knew about this bug class.

IDOR is treated as a beginner's reading bug. Find an ID, change it, see someone's email.

In this dataset, eight of seventeen reports are unauthorized writes — deleting certifications, destroying campaigns, editing profiles, wiping entire accounts. And the references being swapped live overwhelmingly in request bodies: GraphQL variables, JSON payloads, legacy form fields. If you're only fuzzing URL parameters, you're hunting where the bugs used to be.

One more wrinkle.

The biggest bounty in the set — **\$10,500 to a high-severity PayPal report** — wasn't an account takeover. And a medium-severity Shopify report paid \$5,000, while the five criticals show \$0 in the public bounty column.

Money doesn't follow the severity label. It follows what the bug lets you touch.

Where these IDORs actually live

Before the mechanisms, the map. Across the 17 reports, four surfaces showed up again and again:

GraphQL mutations and queries. Seven reports. The object reference is a base64-encoded global ID (`Z2lkOi8vaGFja2Vyb25lL0NhbnBhaWduLzI0NA==`) or an opaque hash inside the `variables` block. The UI never shows it; Burp does.

Legacy form endpoints. IntenseDebate's `hidBlogID`, CrowdSignal's `invite-user.php?id=`, TikTok's `draft_order_id`. Old PHP-style parameters on endpoints nobody redesigned. Sequential, numeric, predictable.

Search and export endpoints. HackerOne's `/bugs.json`, Reddit's mod-log query, Shopify's invoice download, Unikrn's transaction history. Read paths that return data in bulk — the highest-volume leaks in the set.

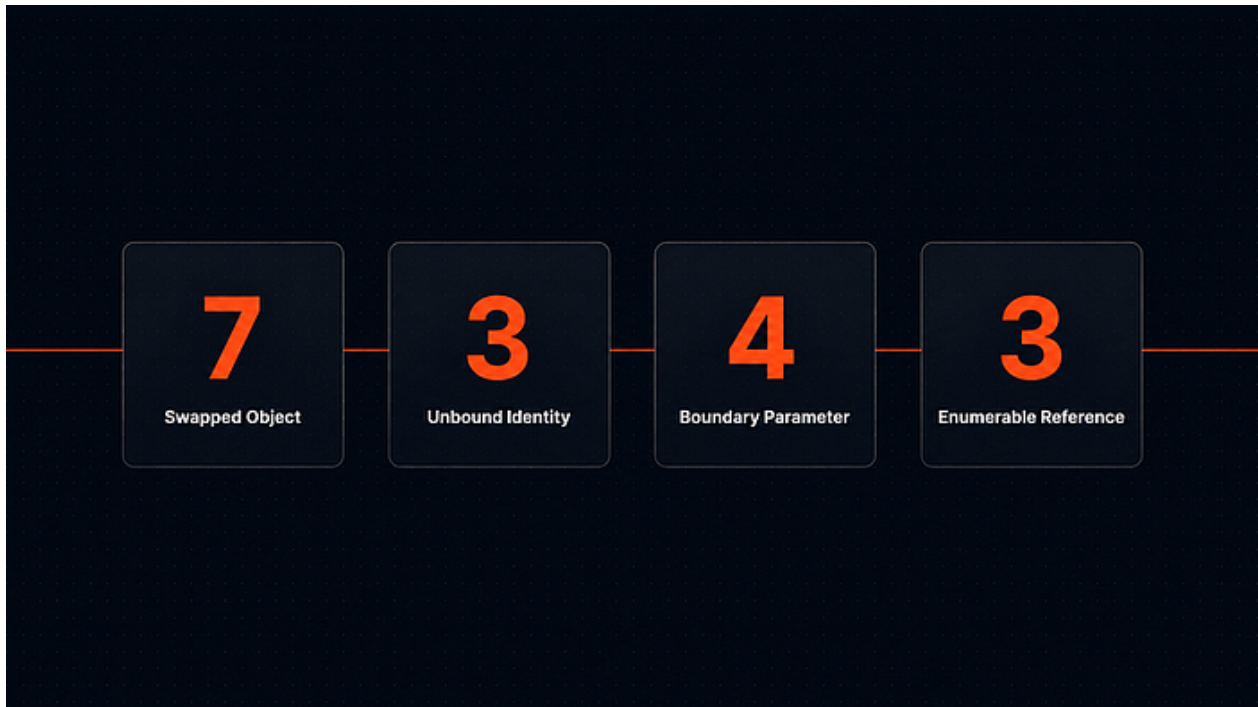
Checkout and account-management flows. Yelp's payment step, Mozilla's account deletion, PayPal's secondary-user management, Starbucks' shared session across sibling domains. Where the money and the identity live.

Notice what's missing: barely any URL path parameters. The bug moved into the body of the request. Your proxy history is the attack surface now.

The four mechanisms

I read all 17 reports and sorted every bug by how the authorization failed — not by program, not by impact. Four mechanisms cover all 17. Not five. Not ten. Four.

#	Mechanism	The failure	Reports
1	Your request, their object	The action is authorized, the object is not	7
2	Two identifiers, never compared	Session says who you are, body says who to act on	3
3	The boundary is a parameter	The scope itself arrives as input	4
4	The reference that scales	IDs predictable enough to enumerate	3



Mechanism 1: Your request, their object

The most common failure in the dataset — seven of 17. The server correctly checks that you're logged in and allowed to run the operation. It never checks that the object inside the operation belongs to you.

The certification wipe. A researcher edits their own Licenses and certifications on hacker.com, intercepts the GraphQL mutation, and changes one number - the ID of the certification record.

Point it at another user's ID and send. The other user's certification is deleted.

Change the ID across a range and you can delete every certification on the platform. ([HackerOne #2122671](https://hackerone.com/reports/2122671) (<https://hackerone.com/reports/2122671>), 388 upvotes, high)

The same shape appears over and over:

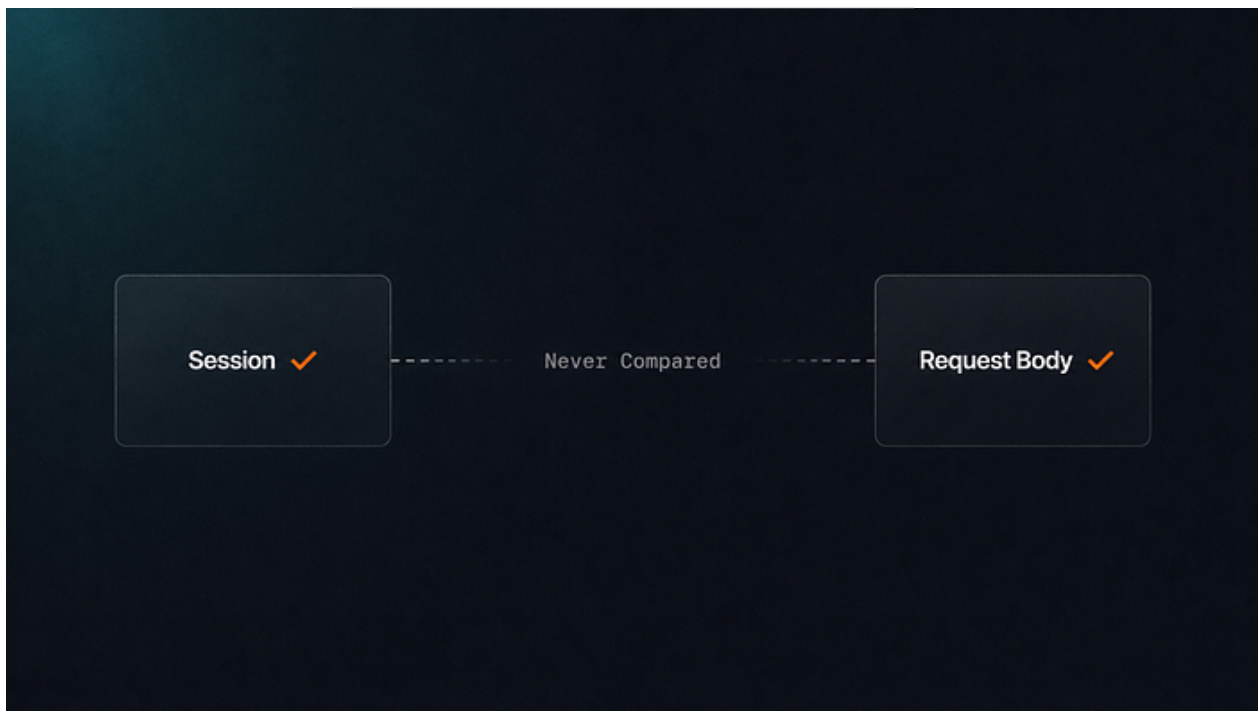
- **Campaign delete** — `UpdateCampaign` mutation, swap the base64 `campaign_id` to any campaign, including programs you don't belong to. ([HackerOne #1969141](https://hackerone.com/reports/1969141) (https://hackerone.com/reports/1969141), 346 upvotes)
- **Ticket delete** — `draft_order_id` on `ads.tiktok.com` deletes other users' support tickets. ([TikTok #1475520](https://hackerone.com/reports/1475520) (https://hackerone.com/reports/1475520), 216 upvotes)
- **Conversation delete** — `DestroyLlmConversation` on an unreleased HackerOne Copilot feature, found by watching the site's JavaScript for new GraphQL operations. ([HackerOne #2218334](https://hackerone.com/reports/2218334) (https://hackerone.com/reports/2218334), 208 upvotes)
- **Blog hijack** — the legacy `hidBlogID` form field on `IntenseDebate` swaps whose blog you're editing. ([Automatic #974222](https://hackerone.com/reports/974222) (https://hackerone.com/reports/974222), 176 upvotes)
- **Tag attach** — `AddTagToAssets` takes a `tagId`, decodes to `gid://hackerone/AsmTag/4979xxxx`, and can be brute-forced to touch other users' assets. ([HackerOne #2633771](https://hackerone.com/reports/2633771) (https://hackerone.com/reports/2633771), 172 upvotes)
- **The Yelp checkout** — `credit_card_id` at `/checkout/transaction_platform`. ([Yelp #391092](https://hackerone.com/reports/391092) (https://hackerone.com/reports/391092))

Why it works. The authorization question the server answers is "may this session run `updateCampaign`?" The question it never asks is "does this campaign belong to this session?" One question validates the verb, nobody validates the noun.

How to test it. Two accounts, one proxy. Run every write operation the app offers — edit, delete, update, assign, tag — as User A against objects created by User B. Don't touch the URL. Swap only the object reference inside the body. If you get a `200 OK` and User B's object changed, you have Mechanism 1.

Mechanism 2: Two identifiers, never compared

Three reports, and the subtlest failure in the dataset. The server holds two facts that describe the request — who you are, and who the request is about — and never checks that they describe the same person.



The account deletion that ignored the password. Firefox Accounts' `POST /v1/account/destroy` requires your email and `authPW` — a hash of your password — in the JSON body. A researcher logged into their own account, started the deletion flow, and captured the request. Then, from the attacker's session, they replayed it with the victim's email and `authPW` in the body.

```
{
  "email": "victims344@gmail.com",
  "authPW": "42b4c2940fe2efecce851a2d8e9754d0f1cb1d37e3ccaabb060f9ac21900caff"
}
```

The server deleted the victim's account. It held the victim's own password hash in the request and still never asked whether the session belonged to the victim. ([Mozilla #3154983](https://hackerone.com/reports/3154983) (https://hackerone.com/reports/3154983), 235 upvotes, high)

- **Cross-domain sessions** — an alternate Starbucks Singapore site shared a database and cookie credentials with `card.starbucks.com.sg`. Copy a `PHPSESSID` from the sibling site, and you're in: user data, password reset, full account takeover. ([Starbucks #876300](https://hackerone.com/reports/876300) (https://hackerone.com/reports/876300), 259 upvotes, critical)
- **Secondary user injection** — a PayPal business account owner could add another account's users as secondary users on their own business, inheriting those users' access. The API validated the caller and the operation, never the relationship between them. ([PayPal #415081](https://hackerone.com/reports/415081) (https://hackerone.com/reports/415081), 786 upvotes, \$10,500 — the largest bounty in this dataset)

Why it works. Authentication and target selection arrive through independent channels — session cookie, JSON body, sibling domain, account hierarchy. Each channel is verified alone. Nothing binds them together.

How to test it. List every identifier in a request: session, cookie, body fields, headers. Ask which one selects the target. Then make them disagree — attacker's session, victim's target — and send. The password hash being correct is not the same as the session being authorized. That sentence is worth re-reading.

Mechanism 3: The boundary is a parameter

Four reports. The endpoint takes the trust boundary itself — the organization, the subreddit, the shop — as an input parameter, and authentication is the only gate.

Private reports, one POST away. HackerOne's `/bugs.json` search endpoint takes `organization_id` and `text_query`. Any logged-in user could POST with an org ID and a single digit, and the search would return that org's private reports — titles, states, severity. Being a member of the org was never checked; the org was just a value in the form body. ([HackerOne #2487889](https://hackerone.com/reports/2487889) (<https://hackerone.com/reports/2487889>), 255 upvotes, critical)

- **Mod logs by name** — Reddit's GraphQL takes `subredditName` for any public or restricted subreddit and returns its moderator logs. No check that you moderate it. Pagination via `after / endCursor` walks the entire log. ([Reddit #1658418](https://hackerone.com/reports/1658418) (<https://hackerone.com/reports/1658418>), 157 upvotes, \$5,000)
- **Other merchants' invoices** — Shopify's `BillDetails` and `BillingDocumentDownload` queries take a `BillingInvoice` id and return another merchant's email, full address, invoice contents, and the last four digits of their card. Severity: medium. Bounty: **\$5,000**. ([Shopify #2207248](https://hackerone.com/reports/2207248) (<https://hackerone.com/reports/2207248>), 180 upvotes)
- **Another customer's wallet** — Unikrn's `transaction-history` handshake leaks another customer's data, enabling enumeration of emails and phone numbers. ([Unikrn #1966006](https://hackerone.com/reports/1966006) (<https://hackerone.com/reports/1966006>), 229 upvotes, \$3,000)

Why it works. The developer treats the scope selector as routing data — "which dataset to query" — not as a security decision. But which dataset is the entire security decision. Authentication proves you're a customer. It says nothing about which customer.

How to test it. Find every request where a parameter narrows the result set: org IDs, team slugs, tenant names, workspace IDs, account numbers. Change it to a value you have no right to. If the response size changes, keep pulling. Follow the pagination cursor to the last page before you write the report — the total volume is your impact section.

Mechanism 4: The reference that scales

Three reports where the ID format itself was the vulnerability. One swapped ID is luck. An enumerable reference is a weapon.

The invite popup that logs you in. CrowdSignal's `invite-user.php?id=` accepted sequential user IDs across a documented range — `00010006` to `19920500+`. Change the ID, and the edit-users popup returns the victim's email. Click **Update Permissions**, and you're logged into the victim's account. No interaction from the victim, full account takeover, and an ID range that spans the entire user base. ([Automattic #915114](https://hackerone.com/reports/915114) (https://hackerone.com/reports/915114), 202 upvotes, critical)

- **Resolve, then rewrite** — on `gql.reddit.com`, one query takes any `username` and returns that user's profile links with their internal IDs. A second mutation takes those IDs and rewrites the links. You don't need to guess a single identifier; the API resolves it for you. ([Reddit #1661113](https://hackerone.com/reports/1661113) (https://hackerone.com/reports/1661113), 206 upvotes)
- **The hash that was too short** — Semrush's marketplace used a hash function whose output length let references be collided and enumerated. The disclosed summary is one line: "Insecure direct object references in marketplace due to a length restrictions in chosen hashing function." Critical. ([Semrush #837400](https://hackerone.com/reports/837400) (https://hackerone.com/reports/837400), 172 upvotes)

Why it works. Unpredictability was quietly doing the work of authorization. When the reference is sequential, resolvable by name, or short enough to collide, that hidden control collapses — and one bug becomes every object in the database.

How to test it. Decode what you can: base64 GraphQL IDs decode to `gid://...` strings that reveal type and numeric key. Create two objects and diff their IDs to detect sequences. Check whether any endpoint resolves a username, email, or slug into an internal ID. And when the reference looks like a hash, ask how many bits of it you'd actually need to guess.

From one invoice to every account: the escalation ladder

The same broken check produces a \$50 finding or a critical. What moves you up is what the object represents.

Rung	What you touch	Real example	Impact
1 — Read	Another user's data	Unikrn (\$3,000) · Shopify (\$5,000)	Email, phone, address, invoices
2 — Write	Their objects	Certifications wiped · Tickets deleted · Profiles edited	Destruction, defacement, extortion leverage
3 — Money	Payment flows	Yelp (1.5M+ cards)	Transactions on someone else's card
4 — Identity	The account itself	CrowdSignal · Starbucks · Mozilla	Full takeover, account deletion

Two lessons from the bounty column. The Shopify invoice leak paid \$5,000 at medium severity because the data was specific: card digits, addresses, invoice contents. And the PayPal report paid \$10,500 without touching another user's account — it weaponized the account hierarchy. Impact is about what the object unlocks, not the severity label the triager picked.



The report that made it click

I'd read all 17 when Mozilla #3154983 rearranged how I think about this class.

The Firefox Accounts API does account deletion the careful way. It doesn't just take a session — it demands the account's email and `authPW`, a hash of the account's password, in the request body. On paper, that's defense in depth.

The researcher's steps take seven moves, and every single one is ordinary:

1. Log into the victim's test account (their own second account).
2. Start the delete flow at `api.accounts.firefox.com/v1/account/destroy`, capture the request in Burp, and note the body — victim email, victim `authPW`. Cancel it.
3. Log into the attacker account.
4. Start the same delete flow, capture it, send it to Repeater.
5. Replace the body with the victim's captured email and `authPW`.
6. Send.
7. The victim's account is gone — deleted by a session that never belonged to it.

The server had everything required to catch this. The session identified one user. The body identified another. The password hash proved the requester had captured the victim's credentials — itself alarming — but no code path compared the two identities.

That's Mechanism 2 in its purest form, and it's why I stopped thinking of IDOR as "change a number, see a stranger's data." An IDOR is any request where two identifiers could disagree and the server doesn't notice. The number-in-a-URL version is just the one they teach first.

Read the full report: [Mozilla #3154983](https://hackerone.com/reports/3154983) (<https://hackerone.com/reports/3154983>)

5 questions to run on every endpoint

The four mechanisms compress into five questions. Run them on every endpoint you test — especially the ones that look boring.

1. Whose object is in this request?

Two accounts, one proxy. Trigger every operation the app offers as User A against User B's objects. URL untouched — swap the reference in the body: GraphQL variables, JSON fields, form params. Catches Mechanism 1.

2. What identifies the target — and does anything compare them?

List the identifiers in play: session cookie, body fields, headers, tokens. Make them disagree and send. A correct password hash with the wrong session should still fail. When it doesn't, that's Mechanism 2.

3. Is the boundary itself a parameter?

Find the scope selectors: `organization_id`, `subredditName`, `workspace`, `tenant`, `shop`. Point them at a scope you don't belong to, then follow the pagination to the last cursor. That's Mechanism 3, and the total volume is your impact number.

4. Can I predict or resolve the reference?

Base64-decode GraphQL IDs and read the `gid://` inside. Create two objects and diff the IDs for sequences. Look for endpoints that turn a username or email into an internal ID. Short hash? Ask how many bits you'd need. That's Mechanism 4.

5. What does this object unlock one level up?

The reference points at a record — but the record points at an account, a payment method, a permission set. Yelp's `credit_card_id` wasn't data leakage; it was someone else's money. PayPal's user reference wasn't PII; it was another company's access. Climb one level before you write the report.

What to remember

An IDOR isn't a missing check on an ID. It's an authorization system that validates the verb and never validates the noun.

The server asks may this user delete, update, download? — and forgets to ask is this object theirs? Every mechanism in this dataset is a variation of that one blind spot: the swapped object (7), the unbound identities (3), the parameterized boundary (4), the enumerable reference (3).

If you take one hunting habit from this: stop testing URLs and start testing objects. Every write operation, every search endpoint, every scope parameter — two accounts, swap the noun, watch what changes.

Next in the Bug Bounty Playbook series: the top SSRF reports on HackerOne — 313 disclosed reports, where the broken assumption stops being about which object you can touch and becomes where the server is willing to send a request on your behalf.

Sources

All links resolved when I wrote this. Three top-20 reports (Nord Security #751577, Pornhub #380410, Pornhub #681473) no longer allow public access and are excluded from the analysis.

1. PayPal — [IDOR to add secondary users in paypal.com/businessmanage](https://hackerone.com/reports/415081) (https://hackerone.com/reports/415081) — \$10,500, 786 upvotes
2. HackerOne — [Delete all Licenses and certifications via CreateOrUpdateHackerCertification](https://hackerone.com/reports/2122671) (https://hackerone.com/reports/2122671) — high, 388 upvotes
3. HackerOne — [Delete Campaigns via UpdateCampaign](https://hackerone.com/reports/1969141) (https://hackerone.com/reports/1969141) — high, 346 upvotes
4. Starbucks — [Account Takeover via IDOR \(Singapore\)](https://hackerone.com/reports/876300) (https://hackerone.com/reports/876300) — critical, 259 upvotes
5. HackerOne — [Viewing Private Report Details via /bugs.json](https://hackerone.com/reports/2487889) (https://hackerone.com/reports/2487889) — critical, 255 upvotes
6. Mozilla — [Account Deletion via Session Misbinding](https://hackerone.com/reports/3154983) (https://hackerone.com/reports/3154983) — high, 235 upvotes
7. Unikrn — [User enumeration and email/phone disclosure within cashier](https://hackerone.com/reports/1966006) (https://hackerone.com/reports/1966006) — high, \$3,000, 229 upvotes
8. TikTok — [Delete any Tickets on ads.tiktok.com](https://hackerone.com/reports/1475520) (https://hackerone.com/reports/1475520) — high, 216 upvotes
9. Yelp — [Order for free using another user's credit card](https://hackerone.com/reports/391092) (https://hackerone.com/reports/391092) — critical, 213 upvotes
10. HackerOne — [IDOR in unreleased Copilot feature](https://hackerone.com/reports/2218334) (https://hackerone.com/reports/2218334) — medium, 208 upvotes
11. Reddit — [Modify the links of any user](https://hackerone.com/reports/1661113) (https://hackerone.com/reports/1661113) — high, 206 upvotes
12. Automattic — [Account Takeover without User Interaction at CrowdSignal](https://hackerone.com/reports/915114) (https://hackerone.com/reports/915114) — critical, 202 upvotes
13. Shopify — [BillingDocumentDownload and BillDetails leak merchant invoices](https://hackerone.com/reports/2207248) (https://hackerone.com/reports/2207248) — medium, \$5,000, 180 upvotes
14. Automattic — [Edit Anyone's Blogs / Websites](https://hackerone.com/reports/974222) (https://hackerone.com/reports/974222) — high, 176 upvotes
15. Semrush — [IDOR via hash length restriction in marketplace](https://hackerone.com/reports/837400) (https://hackerone.com/reports/837400) — critical, 172 upvotes
16. HackerOne — [AddTagToAssets discloses custom tags](https://hackerone.com/reports/2633771) (https://hackerone.com/reports/2633771) — medium, 172 upvotes
17. Reddit — [Mod logs from any subreddit](https://hackerone.com/reports/1658418) (https://hackerone.com/reports/1658418) — high, \$5,000, 157 upvotes